

DDLlaunch+ (Data-Driven Launcher+)

v2.0.0

A small, native Win32 C++ desktop launcher that lists remote resources (hosts, drives, shell items, URLs) from a JSON file and opens each one with a tool chosen by its **type**. Which tool handles which type is also defined in JSON, so new connection types can be added without recompiling.

Author	Glenn Madine
Version	2.0.0 (release date 09/25/2026)
Language / API	C++17, Win32 API, Common Controls, COM (Shell.Application)
Compiler	Microsoft C++ 19.51 (MSVC)
Dependencies	Windows SDK, nlohmann/json (single header <code>json.hpp</code>), <code>resource.h</code> / <code>DDLlaunch.res</code>
Source file	<code>DDLlaunch.cpp</code> (~1,030 lines)

1. What it does

When started, DDLaunch+ opens a resizable 600 × 400 window with:

- **A report-style list view** with three sortable columns: *Resource (Host, Drive, or Shell Item)*, *Type*, and *Description*.
- **Five buttons** in two rows along the bottom:

Button	Behavior
Launch RDP connection for host not in this list	Starts <code>mstsc.exe</code> with no arguments
Edit <code>actionDefinitions.json</code>	Opens the actions file in Notepad
Edit <code>connections.json</code>	Opens the connections file in Notepad
Run...	Shows the standard Windows Run dialog (same as Win+R)
Exit	Closes the application

Double-clicking a row (or pressing **Enter** on it) launches that resource. The app looks up the row's **Type** (upper-cased) in `actionDefinitions.json` and runs the matching `command` with `args + host` as the parameters. For URL-style commands (`https://`, `http://`, `ftp://`, `file://`, `mailto:`, `tel:`) it instead opens `command + host` as a URL, so the default browser or handler opens it.

Clicking a column header sorts the list by that column (case-insensitive); clicking it again reverses the order. A sort arrow is shown in the header.

Whenever the window regains focus, both JSON files are reloaded from disk, so edits made in Notepad take effect as soon as you switch back — no restart needed.

2. Configuration files

Both files live in a per-user folder:

```
%USERPROFILE%\DDLlaunch\actionDefinitions.json
%USERPROFILE%\DDLlaunch\connections.json
```

If `%USERPROFILE%` is not set, the app falls back to the shell's Profile known folder, and if that also fails, to the folder that contains `DDLlaunch.exe`. The folder and both files are created automatically on first run. Existing files are never overwritten.

2.1 actionDefinitions.json

Must be a top-level JSON **array**. Every entry needs all three keys, or the file fails to load.

Key	Meaning
action	Name matched against a connection's <code>type</code> (the type is upper-cased first, so write actions in UPPER CASE)
args	Text placed before the host on the command line (e.g. <code>" /v:"</code>)
command	Executable path, or a URL prefix such as <code>https://</code>

Default content written on first run:

```
[
  { "action": "RDP",    "args": " /v:", "command": "C:\\Windows\\System32\\mstsc.exe" },
  { "action": "SSH",    "args": " ",   "command": "C:\\Windows\\System32\\OpenSSH\\ssh.exe" },
  { "action": "HTTPS",  "args": "",    "command": "https://" },
  { "action": "HTTP",   "args": "",    "command": "http://" }
]
```

2.2 connections.json

Accepts either a flat array of device objects or an object of named groups (each group is an object or an array of objects). Each device can have `host`, `type`, and `description` (all strings, all optional). The default is an empty array `[]`.

Flat array:

```
[
  { "host": "dc01.ynext.corp", "type": "RDP", "description": "Domain controller" },
  { "host": "piweb.ynext.corp", "type": "HTTPS", "description": "Raspberry Pi web server" },
  { "host": "admin@pi01", "type": "ssh", "description": "Pi shell (type insensitive)" }
]
```

Grouped:

```
{
  "Servers": [
    { "host": "cal.ynext.corp", "type": "RDP", "description": "Certificate Authority" },
  ],
  "Web": { "host": "intranet.ynext.corp", "type": "HTTPS", "description": "Intranet" }
}
```

Group names are not shown in the UI; all devices are merged into one list.

3. Building

Requirements: Visual Studio (MSVC 19.x) with the Windows SDK, `json.hpp` from `nlohmann/json`, and a `resource.h` / `DDLaunch.rc` that defines `IDI_ICON1`.

From a *Developer Command Prompt for VS*:

```
rc DDLaunch.rc
CL /EHsc /W3 /O2 /GL /DUNICODE /D_UNICODE /DNDEBUG /std:c++17 DDLaunch.cpp DDLaunch.res /Fe:DDLaunch.exe /link /SUBSYSTEM:WINDOWS comctl32.lib shell32.lib shlwapi.lib user32.lib gdi32.lib ole32.lib oleaut32.lib
```

Header sort arrows (`HDF_SORTUP` / `HDF_SORTDOWN`) and visual styles need Common Controls v6, so the resource file should embed an application manifest (or see recommendation R6 below).

4. Code architecture

The file is organized top to bottom as follows:

Section	Key functions / types	Purpose
Constants	VERSION, IDC_*, COL_*	Control IDs and column indexes

Section	Key functions / types	Purpose
UTF-8 ↔ UTF-16	Utf8ToWide, WideToUtf8	Convert between JSON (UTF-8) and Win32 wide strings
Action model	struct Action, from_json, to_json	nlohmann/json mapping for action entries
File I/O	ReadFileUtf8	Reads a whole file via CreateFileW/ReadFile (supports Unicode paths)
Action lookup	loadActions, findCommandByAction, findArgsByAction	Parse the actions array and look up by name
Device model	struct Device, LoadConnections	Parse connections (array or grouped object)
Paths	GetUserProfileDir, GetDataDir, Get*JsonPath	Resolve %USERPROFILE%\DDLlaunch with fallbacks
First-run setup	EnsureDataFiles, CreateFileWithDefaultContent	Create folder and seed default JSON (CREATE_NEW, race-safe)
Editing	OpenJsonFileForEditing	Opens a config file in Notepad
Launching	LaunchSomething, OnItemActivated	URL vs. executable dispatch via ShellExecuteW
Run dialog	ShowRunDialog	IShellDispatch::FileRun() via COM
Sorting	SortDevices, UpdateHeaderSortArrow	Column sort with direction toggle and header arrow
UI	PopulateListView, ReloadAllData, WndProc	List view, layout, commands, notifications, focus reload
Entry point	wWinMain	Common controls init, window class, message loop

Launch flow

1. User double-clicks a row or presses Enter → WM_NOTIFY (NM_DBLCLK / NM_RETURN).
2. OnItemActivated upper-cases the device type and looks up command and args.
3. LaunchSomething checks whether command starts with a known URL scheme:
 - **Yes:** ShellExecuteW("open", command + host)
 - **No:** ShellExecuteW("open", command, args + host)

Strengths

- Unicode throughout: wide Win32 APIs, UTF-8 JSON, and Win32 file I/O so profile paths with non-ASCII names work.
- Data-driven: new launch types need only a JSON edit.
- Safe first-run setup: CREATE_NEW never overwrites a file and handles a creation race.
- Hot reload on focus makes the edit-in-Notepad workflow smooth.
- Uses the documented Shell.Application COM object for the Run dialog instead of the undocumented shell32 ordinal 61, and balances CoInitializeEx/CoUninitialize correctly.

- A bad `actionDefinitions.json` keeps the previously loaded actions instead of crashing the app.

5. Analysis: issues and recommendations

Severity: **High** = user-visible bug; **Medium** = incorrect or fragile behavior; **Low** = cleanup / maintainability.

5.1 Bugs

#	Severity	Issue	Recommendation
B1	High	Error-dialog loop. <code>ReloadAllData</code> runs on every <code>WM_ACTIVATE</code> . If a JSON file is invalid (for example saved mid-edit), it shows a message box; closing the box reactivates the main window, which reloads, fails, and shows the box again. The user may be unable to return to Notepad without killing the app.	Only reload when a file's last-write time has changed (track it with <code>GetFileAttributesExW</code>), or suppress reloads while an error box is open.
B2	High	Sort order lost on reload. Every reload (each time the window gets focus, including after launching something) replaces <code>g_devices</code> with file order, but the header arrow still shows the old sort.	After loading, re-apply the current sort (<code>g_sortCol/g_sortAsc</code>) without toggling direction, then populate.
B3	Medium	Unknown types fail silently. If a type has no matching action, <code>cmd</code> and <code>args</code> become "Default", so <code>ShellExecuteW</code> tries to run a program named <code>Default</code> . The return value is ignored, so nothing visible happens.	Check the <code>optional</code> results and show a message such as "No action defined for type X". Check the <code>ShellExecuteW</code> return value (≤ 32 means failure).
B4	Medium	file:// check is off by one. <code>prefix.substr(0, 8) == L"file://"</code> compares 8 characters with a 7-character string, so it only matches when the command is exactly <code>file://</code> (not <code>file:///...</code>).	Use a starts-with helper, or detect any scheme generically with <code>PathIsURLW / UrlIsW</code> or by finding <code>://"</code> . This also lets custom schemes such as <code>vnc://</code> or <code>ssh://</code> work.
B5	Medium	Case-sensitive action names. Types are upper-cased but action names are not, so an action written as <code>"Rdp"</code> never matches.	Upper-case both sides (or use <code>_wcsicmp</code>) in the lookup.
B6	Low	Tab key does nothing. Buttons have <code>WS_TABSTOP</code> , but the message loop does not call <code>IsDialogMessageW</code> , so keyboard navigation between controls does not work.	Call <code>IsDialogMessageW(hWnd, &msg)</code> in the loop and add <code>WS_EX_CONTROLPARENT</code> to the main window.
B7	Low	Double load at startup. Data is loaded in <code>WM_CREATE</code> and again on the first <code>WM_ACTIVATE</code> .	Fixed as a side effect of B1's timestamp check.
B8	Low	Connections cleared on read error. If <code>connections.json</code> can't be read, the list is emptied, while a bad actions file keeps the old actions.	Keep the old device list on failure, matching the actions behavior.

5.2 Dead or duplicated code

#	Item	Notes
D1	UsernameDialog struct and GetUsernamePopup()	Never called. GetUsernamePopup also declares unused locals (dlgBase, dlgLayout) and has leftover design comments. Remove, or wire into the SSH action (e.g. prompt for user@host).
D2	class Logger	Never instantiated. Remove, or use it for launch/error logging.
D3	getExeDirectory() vs GetExeDir()	Two functions do the same job; only GetExeDir() is used.
D4	ToWide() vs Utf8ToWide(); WStringToString() vs WideToUtf8()	Duplicate conversion helpers; WStringToString is unused.
D5	findCommandByAction + findArgsByAction	Two linear searches for the same entry. Replace with one findAction() returning const Action*.
D6	PopulateListView(hLV, actions)	The actions parameter is not used.
D7	to_json(Action)	Not used (the app never writes actions); harmless.

5.3 Build and robustness notes

#	Item	Recommendation
R1	#define UNICODE / _UNICODE in the source and /DUNICODE on the command line	May cause warning C4005 (macro redefinition). Use #ifndef UNICODE guards.
R2	toupper and uintptr_t rely on transitive includes	Add <cwctype> and <cstdint>.
R3	Fixed pixel sizes, no DPI awareness	Declare Per-Monitor V2 DPI awareness in the manifest and scale sizes with GetDpiForWindow.
R4	No minimum window size	Handle WM_GETMINMAXINFO so the buttons (which need ~520 px) don't overlap when the window is narrowed.
R5	Stale comments	OpenJsonFileForEditing mentions "the exe directory"; WM_SIZE comments describe an older button layout.
R6	Common Controls v6 depends on the .res manifest	Add #pragma comment(linker, "/manifestdependency:\"type='win32' name='Microsoft.Windows.Common-Controls' version='6.0.0.0' processorArchitecture='*' publicKeyToken='6595b64144ccf1df' language='*'\") so sort arrows always render.
R7	Host values are appended unquoted	A host containing spaces is split into separate arguments. Quote when needed.
R8	Window position and size are not remembered	Optionally save them to a small settings file in the data folder.

5.4 Security note

DDLanch+ runs whatever `command` the JSON says, with the current user's rights. That is by design, and the files sit in the user's own profile, so the risk is the same as a user shortcut. Don't point DDLanch+ at config files in shared or writable-by-others locations.

6. Suggested next steps

1. Fix B1 and B2 together by adding a file-timestamp check and re-applying the sort after reload.
 2. Add a clear error message for unknown types and failed launches (B3).
 3. Replace the hard-coded scheme list with generic URL detection (B4).
 4. Remove the dead code in section 5.2, or finish the SSH username prompt.
 5. Add `IsDialogMessageW`, a manifest dependency, and DPI awareness for a more polished UI.
-

Documentation generated 09/25/2026 from `DDLanch.cpp` v2.0.0.